Proyecto Final

Análisis de los hábitos de escucha en Spotify

Técnicas de Ciencia de Datos
Universidad de la Rioja

Realizado por:

Pablo Galilea Valverde
Jorge Jiménez Sáenz de Tejada

Logroño, Año 2026

Índice general

1. Planteamiento del problema **1**
0.1. Contexto . . . 1
0.2. Análisis de negocio . . . 1
0.3. Objetivo del proyecto . . . 1

2. Datos recogidos **3**
0.4. Fuentes de datos . . . 3
0.5. Acceso a los datos . . . 3
0.5.1. Autenticación en Spotify . . . 3
0.5.2. Autenticación en Last.fm . . . 3
0.5.3. Gestión de credenciales . . . 3
0.6. Datos obtenidos . . . 4
0.7. Problemas de calidad e integración detectados . . . 4

3. Tratamiento de los datos **6**
0.8. Herramientas y librerías utilizadas . . . 6
0.9. Formato de los datos . . . 6
0.10. Proceso de obtención y combinación . . . 6
0.11. Decisiones de diseño . . . 7

4. Diseño del modelo de datos **8**
0.12. Estructura de partida . . . 8
0.13. Enriquecimiento con fuentes adicionales . . . 8
0.14. Tratamiento de valores nulos . . . 9
0.15. Características a estudiar . . . 9
0.16. Líneas de trabajo planteadas . . . 9
0.16.1. Compatibilidad musical entre los dos usuarios . . . 10
0.16.2. Afinidad usuario–artista (recomendación por similitud) . . . 10
0.16.3. Agrupamiento (*clustering*) . . . 10

5. Resultados y visualización **11**
0.17. Plan de visualizaciones . . . 11
0.18. Figuras y tablas de resultados . . . 12

6. Conclusiones **13**
0.19. Conclusiones sobre el proceso de obtención de datos . . . 13
0.20. Conclusiones del análisis . . . 13
0.21. Limitaciones y trabajo futuro . . . 13

Planteamiento del problema

Contexto

Spotify es una de las plataformas de streaming musical con mayor número de usuarios a nivel mundial. A través de su API pública (*Spotify Web API*) permite, previa autenticación del propio usuario, consultar información agregada sobre sus hábitos de escucha: artistas más reproducidos, canciones favoritas, listas de reproducción, etc.

Este proyecto nace con el objetivo de aplicar el ciclo de vida de un proyecto de ciencia de datos — presentado en la primera sesión de la asignatura — sobre un conjunto de datos propio y personal: el historial de artistas más escuchados de los autores de este trabajo, enriquecido con información de género musical obtenida de una segunda fuente (Last.fm).

Análisis de negocio

Siguiendo el esquema de preguntas planteado en la asignatura para delimitar el objetivo de un proyecto de ciencia de datos (¿cuánto?, ¿qué categoría?, ¿qué grupo?, ¿es esto raro?, ¿qué opción se debe tomar?), este trabajo se sitúa principalmente en torno a las preguntas de **categorización** y **agrupamiento**:

- ¿Qué géneros musicales predominan en los gustos de cada usuario?
- ¿Es estable el gusto musical en el tiempo, o varía al comparar el corto plazo (*short_term*, últimas 4 semanas), el medio plazo (*medium_term*, últimos 6 meses) y el largo plazo (*long_term*, varios años)?
- ¿Existen agrupaciones naturales de artistas o géneros que permitan describir el perfil musical de un usuario de forma más compacta que una simple lista?
- ¿Qué diferencias y similitudes existen entre los perfiles musicales de los dos autores del trabajo?

Objetivo del proyecto

El objetivo principal es construir, a partir de datos personales obtenidos mediante la API de Spotify, un conjunto de datos estructurado que recoja los artistas más escuchados por cada autor en los tres rangos temporales disponibles, junto con los géneros musicales asociados a cada artista. Sobre ese conjunto de datos se realizará:

- Un análisis exploratorio (distribución de géneros, comparación entre rangos temporales, comparación entre usuarios).
- Una visualización de los resultados mediante los tipos de gráficos estudiados en la asignatura.
- Opcionalmente, una tarea de agrupamiento (*clustering*) de artistas o géneros, en la línea de lo trabajado en la práctica de aprendizaje no supervisado de la asignatura.

Estado del proyecto

Este es un **primer borrador** del informe. La fase de obtención de datos (Capítulo 0.3 y 0.7) está completada y documentada. Las fases de diseño del modelo de datos, análisis, visualización de resultados y conclusiones (Capítulos 0.11, 0.16.3 y 0.18) se han dejado planteadas a modo de esquema, pendientes de completar con el análisis definitivo sobre los datos ya recogidos.

Datos recogidos

Fuentes de datos

Siguiendo la clasificación de fuentes públicas de datos vista en la asignatura, este proyecto utiliza dos fuentes encuadradas en la categoría de **APIs y datos en tiempo real / redes sociales**:

- **Spotify Web API**, a través de la librería `spotipy` para Python. Es la fuente primaria: proporciona, para el usuario autenticado, su lista de artistas más escuchados (`current_user_top_artists`) en tres rangos temporales.
- **Last.fm API** (`artist.gettoptags`), utilizada como fuente secundaria para enriquecer cada artista con sus géneros musicales más representativos, ya que la API de Spotify para usuarios individuales no siempre ofrece esta información con el mismo nivel de detalle que Last.fm.

A diferencia de los portales de datos abiertos gubernamentales o los repositorios de *Machine Learning* vistos en la Sesión 1 (Kaggle, UCI, datos.gob.es, etc.), estas dos fuentes requieren autenticación mediante credenciales de desarrollador (*API keys* / OAuth) y devuelven datos **personales y dinámicos**: el resultado depende del usuario autenticado y cambia con el tiempo según sus hábitos de escucha.

Acceso a los datos

Autenticación en Spotify

El acceso a los datos de Spotify requiere registrar una aplicación en el panel de desarrolladores de Spotify para obtener un `Client ID` y un `Client Secret`, y autenticar al usuario mediante el flujo OAuth *Authorization Code Flow*, gestionado por `spotipy.oauth2.SpotifyOAuth`. El *scope* solicitado es `user-top-read`, que da acceso de solo lectura a los artistas y canciones más escuchados por el usuario.

Autenticación en Last.fm

El acceso a Last.fm se realiza mediante una `API key` simple, enviada como parámetro en cada petición HTTP al *endpoint* `artist.gettoptags`.

Gestión de credenciales

Las credenciales (`SPOTIPY_CLIENT_ID`, `SPOTIPY_CLIENT_SECRET`, `SPOTIPY_REDIRECT_URI` y `LASTFM_API_KEY`) se almacenan en un fichero `.env` local, cargado mediante la librería `python-dotenv`, y no se incluyen en el repositorio del proyecto por motivos de seguridad.

Datos obtenidos

Para cada uno de los dos autores del trabajo se ha ejecutado el proceso de extracción, obteniendo un listado de hasta 50 artistas por cada uno de los tres rangos temporales que ofrece la API de Spotify:

Rango (API)	**Significado**	**Etiqueta usada**
`short_term`	Últimas 4 semanas	`ultimas_4_semanas`
`medium_term`	Últimos 6 meses	`ultimos_6_meses`
`long_term`	Varios años (histórico acumulado)	`varios_anios`

Cuadro 1: Rangos temporales proporcionados por la API de Spotify para el ranking de artistas más escuchados.

Para cada artista se recogen los siguientes campos:

Campo	**Descripción**	**Fuente**
`rango`	Rango temporal al que pertenece la entrada (ver Tabla 1)	Spotify
`posicion`	Posición en el ranking dentro de su rango (1–50)	Spotify
`artista`	Nombre del artista	Spotify
`generos`	Hasta 4 etiquetas de género, separadas por `;`	Last.fm
`spotify_url`	Enlace al perfil del artista en Spotify	Spotify

Cuadro 2: Esquema del conjunto de datos recogido por autor (fichero `top_artistas*.csv`).

A modo de ejemplo, un artista puede aparecer repetido en los tres rangos (si lleva tiempo siendo escuchado de forma constante) o solo en uno de ellos (si es un descubrimiento reciente o un gusto puntual del pasado), lo cual es en sí mismo una variable de interés para el análisis de la estabilidad del gusto musical.

Problemas de calidad e integración detectados

- **Cobertura incompleta de géneros**: no todos los artistas tienen etiquetas en Last.fm, especialmente bandas sonoras de videojuegos o artistas japoneses minoritarios (p. ej. `ATLUS GAME MUSIC` o compositores japoneses con pocos oyentes en Last.fm), que aparecen sin género asignado.
- **Etiquetas de Last.fm ruidosas**: Last.fm permite etiquetas libres de usuario, por lo que junto a géneros reales aparecen etiquetas de nacionalidad (`spanish`,

`japanese`, `taiwan`) o metaetiquetas sin valor musical (`seen live`, `favorites`, `my top songs`). Se ha aplicado un filtro manual (`IGNORAR`) para excluir las etiquetas más comunes de este tipo, aunque no es exhaustivo.

- **Redundancia entre rangos**: un mismo artista puede aparecer hasta tres veces (una por rango), por lo que el fichero combinado no está en forma de tabla *artista–atributos* única, sino en formato largo (*long format*), con una fila por combinación artista-rango.
- **Normalización de nombres**: los nombres de artistas provienen directamente de Spotify y se usan tal cual para consultar Last.fm; en caso de alias distintos entre ambas plataformas, la búsqueda de género puede fallar silenciosamente (se guarda cadena vacía).

Tratamiento de los datos

Herramientas y librerías utilizadas

El tratamiento de los datos se ha realizado íntegramente en Python, dentro de un cuaderno Jupyter (`TrabajoTecnicasDeCienciaDeDatos.ipynb`), utilizando las siguientes librerías:

Librería	**Uso en el proyecto**
`spotipy`	Cliente Python para la Spotify Web API; gestiona el flujo OAuth y las peticiones a los *endpoints* de artistas más escuchados.
`requests`	Peticiones HTTP directas al *endpoint* REST de Last.fm (`artist.gettoptags`).
`pandas`	Construcción del `DataFrame` con los datos combinados y exportación a CSV.
`python-dotenv`	Carga de credenciales desde el fichero local `.env`, evitando guardarlas en el código fuente.
`matplotlib`	Prevista para la fase de visualización (Capítulo 0.16.3); instalada desde el inicio del proyecto.
`time` (estándar)	Introduce una pausa (`time.sleep(0.25)`) entre peticiones a Last.fm para respetar el límite de peticiones de su API.

Cuadro 3: Librerías empleadas en la fase de obtención y tratamiento de datos.

Formato de los datos

Se ha optado por **CSV** (valores separados por comas) como formato de intercambio y almacenamiento, codificado en `utf-8-sig` para preservar correctamente los caracteres no latinos presentes en los datos (nombres de artistas en japonés, coreano, etc.) y mantener compatibilidad con Excel. CSV es un formato de datos estructurado y en texto plano, adecuado para un conjunto de datos de tamaño reducido (decenas de filas) que se procesará posteriormente con `pandas`, frente a alternativas más orientadas a grandes volúmenes o datos jerárquicos como HDF5 (vista en la Sesión 3 de la asignatura), que no se justifican para este volumen de datos.

Proceso de obtención y combinación

El proceso implementado sigue los siguientes pasos:

1. **Autenticación**: se cargan las credenciales desde `.env` y se inicializa el cliente `spotipy.Spotify` con el *scope* `user-top-read`.

2. **Descarga de artistas por rango**: para cada uno de los tres rangos temporales (Tabla 1), se solicita a la API de Spotify el *top* 50 de artistas más escuchados (`current_user_top_artists`) y se añade cada uno a una lista de registros con su posición, nombre y enlace de Spotify.

3. **Deduplicación antes de consultar géneros**: dado que un mismo artista puede repetirse en varios rangos, se extrae el conjunto de artistas *únicos* (`df["artista"].unique()`) antes de consultar Last.fm, para minimizar el número de peticiones a la API externa y respetar sus límites de uso.

4. **Consulta de géneros en Last.fm**: para cada artista único se solicita el *endpoint* `artist.gettoptags`, que devuelve etiquetas junto con un peso (*count*) entre 0 y 100. Se seleccionan como máximo 4 etiquetas con peso igual o superior a 20, descartando las etiquetas no musicales recogidas en el conjunto `IGNORAR` (nacionalidades, tipo de voz, etc.).

5. **Combinación**: los géneros obtenidos se incorporan al `DataFrame` original mediante un *mapeo* artista → géneros (`df["generos"] = df["artista"].map(cache)`), evitando así repetir la consulta a Last.fm para un mismo artista que aparece en varios rangos.

6. **Exportación**: el resultado final se guarda como `top_artistas.csv`, con las columnas `rango`, `posicion`, `artista`, `generos` y `spotify_url` (Tabla 2).

Decisiones de diseño

- Se ha mantenido una primera versión simplificada del script (sin consulta a Last.fm) junto a la versión final, documentando así la evolución del proceso de obtención de datos: primero se validó el acceso a la API de Spotify y, una vez funcionando, se añadió el enriquecimiento con géneros.

- El límite de **4 géneros por artista** y el **umbral mínimo de peso 20** se han fijado de forma manual para equilibrar la riqueza de información (más géneros) con la precisión (evitar etiquetas minoritarias o poco representativas).

- Cada autor ejecuta el proceso de forma independiente sobre su propia cuenta de Spotify, generando un CSV por persona (p. ej. `top_artistasJorge.csv`), lo que permitirá posteriormente comparar los perfiles musicales de ambos.

Diseño del modelo de datos

Pendiente de completar

Esta sección recogerá el diseño final del modelo de datos sobre el que se realizará el análisis, una vez decididas las características concretas a estudiar. A continuación se deja el esquema de trabajo previsto.

Estructura de partida

El conjunto de datos de partida (Capítulo 0.3) está en **formato largo**: una fila por combinación (artista, rango) y usuario. Para el análisis se plantea derivar de él las siguientes vistas:

- **Tabla artista–género (desagregada)**: una fila por cada género individual de cada artista, separando la columna `generos` (actualmente una cadena con géneros unidos por `;`) en varias filas. Necesaria para contar frecuencias de género.
- **Tabla por usuario y rango**: agregación del número de artistas y de géneros distintos por usuario y por rango temporal, para comparar la diversidad musical de cada autor y su evolución en el tiempo (corto, medio y largo plazo).
- **Tabla combinada de ambos usuarios**: unión de los CSV de cada autor añadiendo una columna `usuario`, para poder comparar directamente sus perfiles.

Enriquecimiento con fuentes adicionales

Se plantea ampliar el conjunto de datos actual (artista, géneros, posición, rango) con más atributos por artista, de forma que el modelo de datos no se limite al género. Las fuentes candidatas, todas accesibles sin necesidad de renegociar permisos adicionales en Spotify, son:

- **Last.fm** – `artist.getInfo`: número de oyentes (*listeners*) y reproducciones totales (*playcount*) del artista, como *proxy* de su popularidad general.
- **Last.fm** – `artist.getSimilar`: lista de artistas similares a uno dado, cada uno con un *match score* entre 0 y 1. Es la pieza clave para la línea de trabajo de afinidad usuario–artista (§0.16.2).
- **Deezer API**: alternativa pública y sin autenticación compleja para contrastar popularidad y metadatos básicos del artista.
- **MusicBrainz**: metadatos canónicos (país de origen, año de actividad, etc.) útil para desambiguar artistas con nombres parecidos entre fuentes.

Nótese que la *Spotify Web API* dejó de ofrecer, desde noviembre de 2024, los *endpoints* de `audio-features` y `related-artists` para aplicaciones de desarrollador nuevas, por lo que las características de audio (p. ej. *danceability*, *energy*) y los artistas relacionados de Spotify no están disponibles como fuente para este proyecto; de ahí que se recurra a Last.fm como sustituto para la señal de similitud entre artistas.

Tratamiento de valores nulos

El conjunto de datos actual ya presenta valores nulos (cadena vacía en `generos`) para los artistas sin etiquetas en Last.fm, descrito en el Capítulo 0.3. Al incorporar los nuevos atributos de §0.13 se espera que la proporción de nulos aumente, ya que no todos los artistas –especialmente los menos conocidos o de nicho (bandas sonoras, artistas japoneses/coreanos minoritarios)– estarán presentes en todas las fuentes. Se plantea:

- Cuantificar el porcentaje de nulos por columna y por fuente, antes de decidir cómo tratarlos.
- Para atributos numéricos (oyentes, *playcount*): valorar imputación por mediana o por grupo (p. ej. por género principal), frente a dejarlos como nulo si el porcentaje de huecos es muy alto.
- Para el atributo de similitud (`artist.getSimilar`): un artista sin vecinos similares conocidos no se imputa, se trata como información ausente real, ya que no tiene un sustituto razonable.
- Documentar siempre qué artistas quedan sin completar tras el proceso, en lugar de descartarlos silenciosamente.

Características a estudiar

- Frecuencia de cada género musical, global y por usuario.
- Grado de coincidencia de artistas entre los tres rangos temporales (estabilidad del gusto musical).
- Grado de solapamiento entre los perfiles musicales de los dos usuarios (artistas y géneros en común).
- Posición en el *ranking* como indicador de intensidad de escucha (los primeros puestos de cada rango representan los artistas más escuchados).
- Popularidad del artista (oyentes/*playcount* de Last.fm) frente a su posición en el *ranking* personal, para ver si los autores escuchan más artistas populares o más de nicho.

Líneas de trabajo planteadas

A partir de los datos disponibles y de los enriquecimientos previstos, se plantean tres líneas de análisis complementarias. Todas ellas son ideas de partida y podrán ajustarse conforme avance el proyecto.

Compatibilidad musical entre los dos usuarios

Dado que se dispone de un conjunto de datos equivalente para cada autor (`top_artistas.csv` y `top_artistasJorge.csv`), se plantea calcular un **índice de compatibilidad musical** entre ambos, por ejemplo mediante el coeficiente de Jaccard sobre el conjunto de artistas o géneros en común, o mediante la similitud del coseno entre los vectores de frecuencia de género de cada usuario. Este índice podría calcularse también por rango temporal, para ver si la compatibilidad es mayor en el gusto reciente que en el histórico.

Afinidad usuario–artista (recomendación por similitud)

Se plantea explorar si, dado un artista nuevo (no presente en el historial de un usuario), se puede estimar si le gustaría, sin necesidad de un clasificador supervisado clásico. La razón es que no existe una fuente fiable de ejemplos *negativos*: la API de Spotify no expone artistas bloqueados u ocultados por el usuario, y tratar como negativo un artista con menos escuchas o presente solo en `long_term` sería un sustituto débil y discutible. En su lugar, se propone un enfoque basado en **similitud de contenido**:

1. A partir de `artist.getSimilar` (Last.fm), obtener los artistas más cercanos al artista candidato.
2. Calcular qué proporción de esos artistas similares ya están presentes en el historial del usuario (y con qué peso, según su posición en el *ranking*).
3. Usar ese solapamiento como medida de afinidad estimada, en lugar de una predicción binaria estricta de "le gusta / no le gusta".

Agrupamiento (*clustering*)

En línea con lo trabajado en la práctica de aprendizaje no supervisado de la asignatura, se plantea explorar una codificación de cada artista mediante sus géneros (variables *dummy* uno por género) para aplicar un algoritmo de *clustering* (p. ej. K-Means) que agrupe artistas o usuarios por afinidad de género, de forma análoga al análisis de clientes realizado en la Práctica 3 de Aprendizaje Automático.

Resultados y visualización

Pendiente de completar

Este capítulo recogerá las visualizaciones y los resultados numéricos obtenidos al ejecutar el análisis sobre el modelo de datos descrito en el Capítulo 0.11. Se deja planteado el plan de visualizaciones, siguiendo los criterios de audiencia, objetivo y tipo de gráfico vistos en la Sesión 6 de la asignatura (Visualización de datos).

Plan de visualizaciones

Pregunta	**Gráfico previsto**	**Referencia**
¿Qué géneros predominan por usuario?	Gráfico de barras horizontales (*top* N géneros por frecuencia)	§0.15
¿Cómo se reparten los géneros en conjunto?	Gráfico circular o de barras apiladas, comparando ambos usuarios	§0.15
¿Cambia el gusto musical según el rango temporal?	Gráfico de barras agrupado por rango (`countplot` por rango y género)	Tabla 1
¿Cuántos artistas se repiten entre rangos?	Diagrama de Venn o tabla cruzada de coincidencias	§0.15
¿Qué tan compatibles son los dos usuarios?	Gráfico de barras o mapa de calor con el índice de compatibilidad por rango	§0.16.1
¿Artistas populares o de nicho?	*Scatter plot* de posición en el *ranking* frente a popularidad (Last.fm)	§0.13
Afinidad estimada con un artista nuevo	Gráfico de barras con el grado de solapamiento con artistas similares	§0.16.2
Agrupamiento de artistas/usuarios por género (opcional)	*Scatter plot* del resultado del *clustering*, con baricentros	§0.16.3

Cuadro 4: Plan de visualizaciones previsto para el análisis de resultados.

Figuras y tablas de resultados

(Pendiente de incorporar las figuras generadas a partir del análisis sobre `top_artistas.csv` *y* `top_artistasJorge.csv`*.)*

Conclusiones

Pendiente de completar

Las conclusiones definitivas se redactarán una vez completado el análisis y la visualización de resultados (Capítulos 0.11 y 0.16.3). A continuación se dejan algunas conclusiones preliminares, centradas en el propio proceso de obtención de datos.

Conclusiones sobre el proceso de obtención de datos

- Combinar dos APIs externas (Spotify y Last.fm) permite enriquecer un conjunto de datos personal con información que ninguna de las dos fuentes ofrece de forma completa por sí sola, a costa de introducir ruido y huecos (artistas sin género) propios de combinar fuentes de distinta calidad y cobertura.
- El uso de una caché en memoria por artista único, en lugar de consultar Last.fm una vez por cada fila del conjunto de datos, resultó clave para mantener el proceso dentro de los límites de peticiones de la API externa.
- Trabajar con datos personales y dinámicos (dependientes del usuario autenticado y del momento de la consulta) implica que los resultados de este proyecto no son directamente reproducibles por un tercero sin sus propias credenciales y su propio historial de escucha, a diferencia de un conjunto de datos abierto y estático.

Conclusiones del análisis

(Pendiente de redactar tras completar los Capítulos 0.11 y 0.16.3: diversidad de géneros por usuario, estabilidad del gusto musical entre rangos temporales, grado de afinidad musical entre los dos autores y, en su caso, interpretación de los grupos obtenidos mediante clustering*.)*

Limitaciones y trabajo futuro

- El filtro de etiquetas no musicales de Last.fm (`IGNORAR`) es manual y no exhaustivo; podría mejorarse con una lista de géneros de referencia o un modelo de clasificación de etiquetas.
- El análisis se limita a dos usuarios; las conclusiones sobre similitud/diversidad musical no son generalizables más allá de los propios autores.
- La línea de afinidad usuario–artista (§0.16.2) se apoya en similitud de contenido precisamente porque no existe una fuente fiable de ejemplos negativos (Spotify no expone artistas bloqueados/ocultados); un clasificador supervisado real de "le gusta / no le gusta" quedaría fuera del alcance de los datos disponibles.

- La *Spotify Web API* ya no ofrece, para aplicaciones nuevas, los *endpoints* de `audio-features` ni `related-artists` (restringidos desde noviembre de 2024), lo que limita el enriquecimiento a lo que puedan aportar fuentes externas como Last.fm, Deezer o MusicBrainz (§0.13).
- Como ampliación, podría incorporarse también el historial de canciones más escuchadas (no solo artistas) además de los atributos de popularidad y similitud ya planteados.